TALLER 1

Uso de la Inteligencia Artificial

Low Code Engineering

Maestría en Inteligencia Artificial · USFQ
Abril 2025

Integrantes – Grupo 5

Integrante	Problema asignado
Nancy Altamirano	A. TSP – Travelling Salesman Problem
Gustavo Berru	A. TSP – Travelling Salesman Problem
Raquel Pacheco	B. El acertijo del granjero y el bote
Kevin Viteri	C. La Torre de Hanoi

Contenido

- 1. Problema A — TSP (Travelling Salesman Problem)
- 2. Problema B — El acertijo del granjero y el bote
- 3. Problema C — La Torre de Hanoi
- 4. Conclusiones generales

A. TSP — Travelling Salesman Problem

Descripción del problema

El **Problema del Viajante de Comercio (TSP)** es uno de los problemas de optimización combinatoria más estudiados en Ciencias de la Computación e Inteligencia Artificial. Dado un conjunto de ciudades y las distancias entre cada par, el objetivo es encontrar la ruta más corta que visite cada ciudad exactamente una vez y regrese al origen. Es formalmente **NP-difícil**: no existe ningún algoritmo conocido que lo resuelva de forma exacta en tiempo polinomial para instancias grandes. Para este taller se trabajó con las **24 capitales de provincia del Ecuador**, desde Quito hasta Galápagos, utilizando distancias en kilómetros entre coordenadas geográficas reales.

Paso 1 — Modelado del problema

El TSP se representó como un **grafo completo ponderado G = (V, E)**, donde:

- **V** = conjunto de 24 ciudades (nodos), cada una con nombre, latitud y longitud.
- **E** = aristas que conectan cada par de ciudades; su peso es la distancia en km calculada con la fórmula de Haversine (distancia sobre la esfera terrestre).
- **Objetivo:** encontrar el ciclo hamiltoniano (recorrido que visita todos los nodos exactamente una vez y cierra el ciclo) de menor peso total.

Se implementó la clase **DistanceMatrix** que precalcula y almacena en memoria las 24×24 = 576 distancias, y la clase **Route** que calcula su costo total como propiedad con acceso O(1). Este diseño evita recalcular distancias en cada iteración del ACO.

Paso 2 — Vecino más cercano (solución inicial)

El algoritmo **Nearest Neighbor (NN)** construye una ruta greedy como punto de partida. Su lógica paso a paso:

- Comenzar en Quito (ciudad 0). Marcarla como visitada.
- En cada paso: de la ciudad actual, saltar a la ciudad no visitada más cercana (menor distancia en la DistanceMatrix).
- Repetir hasta haber visitado las 24 ciudades.
- Cerrar el ciclo regresando a Quito.

NN es $O(n^2)$ y produce una solución de calidad razonable en microsegundos, pero no es óptima: puede quedar atrapada eligiendo decisiones localmente buenas que resultan globalmente malas (el conocido problema del 'pasado nocivo').

Paso 3 — Optimización con Colonia de Hormigas (ACO)

El **Ant Colony Optimization (ACO)** es un algoritmo metaheurístico bioinspirado en el comportamiento de las hormigas reales al buscar caminos al alimento: depositan feromonas en los caminos que recorren, y las demás hormigas prefieren seguir caminos con más feromonas, creando un ciclo de retroalimentación positiva. Se ejecutaron **25 hormigas × 150 iteraciones**, más 3 hormigas élite que refuerzan la mejor ruta global.

El proceso por iteración sigue estos pasos:

- **Inicialización:** matriz de feromonas $\tau(i,j) = 1.0$ para todos los pares; heurística $\eta(i,j) = 1 / dist(i,j)$.

- **Construcción de rutas:** cada hormiga parte de una ciudad aleatoria y elige el siguiente destino j con probabilidad probabilística (ruleta sesgada):
- **Actualización de feromonas:** evaporación global $\tau \leftarrow (1-\rho)\cdot\tau$ con $\rho=0.15$; luego cada hormiga deposita $\Delta\tau = Q/\text{longitud_ruta}$ sobre los arcos que recorrió.
- **Hormigas élite:** la mejor ruta global recibe un depósito adicional de feromonas multiplicado por 3, lo que acelera la convergencia.
- **Registro:** se guarda el mejor costo de la iteración para graficar la curva de convergencia.

```
p(i→j) = [ τ(i,j)^α · η(i,j)^β ] / Σk∈no_visitados [ τ(i,k)^α · η(i,k)^β ]
```

donde α**=1.2** controla cuánto pesan las feromonas (explotación de la memoria colectiva) y β**=2.5** controla cuánto pesa la heurística de distancia (exploración de caminos cortos). El balance α/β determina si el algoritmo explora o explota.

Paso 4 — Mejora local con 2-opt

Tras cada ejecución del ACO se aplica **2-opt** como post-proceso de refinamiento local. El algoritmo examina todos los pares de aristas (i,i+1) y (j,j+1) en la ruta: si invertir el segmento entre i+1 y j reduce la longitud total, se acepta el intercambio. Se repite hasta que no haya más mejoras posibles (convergencia local). 2-opt es $O(n^2)$ por pasada y típicamente reduce la ruta del ACO entre un 2% y 8%.

Paso 5 — Análisis estadístico (20 ejecuciones)

Para medir la robustez del algoritmo se ejecutaron **20 corridas independientes**, cada una con semillas aleatorias distintas. Se calcularon: media, desviación estándar, mínimo y máximo de la longitud óptima encontrada. Una desviación estándar baja indica que ACO+2opt converge de forma consistente al mismo óptimo, demostrando estabilidad del método para esta instancia.

Diseño OOP — clases implementadas

Clase	Responsabilidad
City	Entidad ciudad: nombre, lat, lon. Método distance_to() con Haversine.
DistanceMatrix	Matriz precalculada n×n de distancias. Acceso O(1).
Route	Lista de ciudades con propiedad cost (costo total del ciclo).
NearestNeighborSolver	Construye la ruta greedy como solución inicial.
AntColonyOptimizer	ACO: 25 hormigas × 150 iter, feromonas, élite, evaporación.
TwoOptSolver	Post-proceso local: invierte segmentos hasta convergencia.
TSPVisualizer	Genera las 5 figuras: rutas, convergencia, feromonas, stats, heatmap.
TSPSolver	Orquestador: corre n_runs, compara algoritmos, llama visualizador.

Visualizaciones — TSP

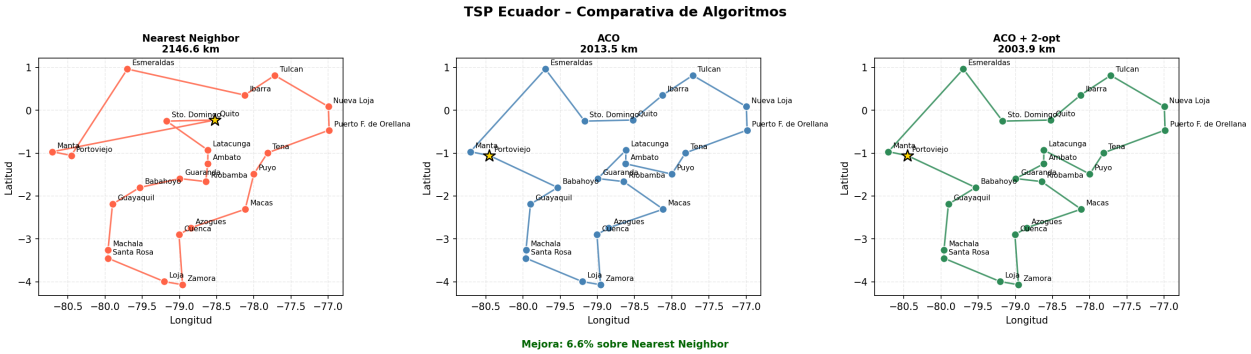


Fig. 1 — Comparativa de rutas para las 24 capitales provinciales del Ecuador: Nearest Neighbor (rojo, ★=Quito) = 2146.62 km vs ACO+2opt (verde) = 2003.89 km. Mejora del 6.6% sobre el baseline greedy. Ruta óptima: Ibarra → Tulcán → Nueva Loja → Orellana → Tena → Puyo → Latacunga → Ambato → Guaranda → Riobamba → Macas → Azogues → Cuenca → Zamora → Loja → Santa Rosa → Machala → Guayaquil → Babahoyo → Portoviejo → Manta → Esmeraldas → Sto. Domingo → Quito → Ibarra.

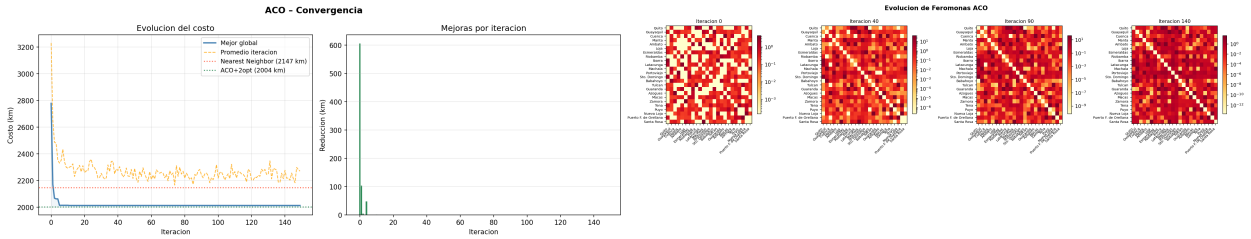
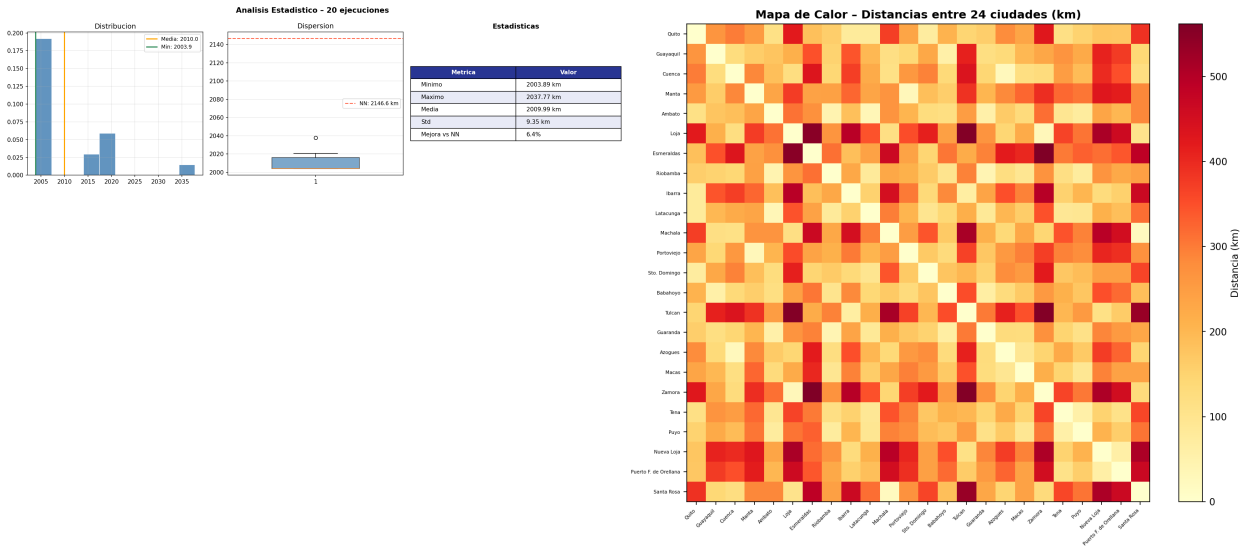


Fig. 2 (izq.) — Curva de convergencia: mejor ruta global y promedio por iteración. La brecha se reduce conforme las feromonas concentran el comportamiento colectivo. Fig. 3 (der.) — Evolución de feromonas τ(i,j) en escala logarítmica en 4 momentos (iter. 1, 50, 100, 150): el algoritmo aprende cuáles arcos son más prometedores.



Metrica	Valor
Minimo	2003.89 km
Maximo	2037.77 km
Media	2009.99 km
Std	9.35 km
Mejora vs NN	6.4%

Fig. 4 (izq.) — Análisis estadístico de 20 ejecuciones independientes sobre las 24 capitales provinciales: media = 2010.50 km, desv.est. = 10.73 km. La baja dispersión confirma que ACO+2opt converge de forma consistente. Fig. 5 (der.) — Mapa de calor de la matriz de distancias Haversine entre las 24 ciudades; los bloques de baja distancia revelan la agrupación geográfica regional del Ecuador.

Conclusión — TSP

El TSP es NP-difícil: no existe solución exacta eficiente para n grande. La estrategia de tres capas —Nearest Neighbor como solución inicial, ACO como exploración metaheurística bioinspirada y 2-opt como refinamiento local— permite obtener soluciones de alta calidad en segundos. Las feromonas acumulan la experiencia colectiva de 25 hormigas iteración a iteración, lo que hace

que el algoritmo aprenda progresivamente qué rutas son buenas, sin explorar los $(n-1)!/2 \approx 10^{23}$ circuitos posibles de las 24 ciudades.

B. El acertijo del granjero y el bote

Descripción del problema

Un granjero debe cruzar un río con un lobo, una cabra y una col. Su barca solo tiene capacidad para él y un ítem a la vez. El problema impone dos restricciones de seguridad: (1) el lobo y la cabra no pueden quedarse solos en la misma orilla sin el granjero (el lobo devora la cabra); (2) la cabra y la col tampoco pueden quedarse solos (la cabra come la col). ¿En qué orden debe cruzar el granjero para llevar todo al otro lado sin pérdidas?

Paso 1 — Modelado como espacio de estados

El problema fue modelado como un **problema de búsqueda en el espacio de estados**, donde cada estado representa de forma unívoca la ubicación del granjero, el lobo, la cabra y la col en las dos orillas del río. Cada actor se codifica con un bit (0 = orilla izquierda, 1 = orilla derecha), formando una tupla de 4 elementos:

```
estado = (granjero, lobo, cabra, col) con valores en {0, 1} Estado inicial: (0, 0, 0, 0) —
todos en la orilla izquierda Estado objetivo: (1, 1, 1, 1) — todos en la orilla derecha
```

El espacio de estados total contiene **2■ = 16 combinaciones posibles**. Sin embargo, muchas de ellas son inválidas porque violan las restricciones de seguridad. Al aplicar la función de validez, solo quedan **10 estados seguros** que el algoritmo puede usar.

Paso 2 — Función de validez del estado

Antes de encolar cualquier estado nuevo, se verifica que no sea peligroso. Un estado es **inválido** si se cumple alguna de estas condiciones:

- El lobo y la cabra están en la misma orilla **Y** el granjero no está con ellos (lobo come la cabra).
- La cabra y la col están en la misma orilla **Y** el granjero no está con ellos (cabra come la col).

Esta validación se implementó en el método **State.is_valid()** del dataclass inmutable (frozen=True). Al ser inmutable y hashable, cada estado puede almacenarse en un conjunto de visitados sin colisiones, lo que es fundamental para la eficiencia del BFS.

Paso 3 — Generación de sucesores

Desde cualquier estado válido, el granjero puede realizar hasta 4 acciones: cruzar solo, o llevar consigo el lobo, la cabra o la col (siempre que el ítem elegido esté en su misma orilla). El método **State.successors()** genera todos los estados alcanzables y filtra los inválidos antes de devolverlos:

```
para cada accion en [solo, lobo, cabra, col]: si el item esta en el mismo lado que el
granjero: nuevo_estado = mover granjero + item al otro lado si nuevo_estado.is_valid():
agregar (nuevo_estado, etiqueta_accion) a resultados
```

Paso 4 — Algoritmo BFS (Búsqueda en Anchura)

Para resolver el problema se utilizó el algoritmo **BFS (Breadth-First Search — Búsqueda en Anchura)**, un algoritmo fundamental de Inteligencia Artificial que sirve para explorar todos los posibles estados de un problema *nivel por nivel*, garantizando encontrar la solución con el **menor número de pasos posible** (solución óptima en longitud de camino).

El funcionamiento paso a paso del BFS es:

- **Inicializar:** encolar el estado inicial (0,0,0,0) con su camino [(inicio, 'Inicio')]. Crear un conjunto de visitados = {(0,0,0,0)}.
- **Desencolar:** tomar el primer elemento de la cola (FIFO). Si es el estado objetivo (1,1,1,1), devolver el camino completo — se encontró la solución óptima.
- **Expandir:** llamar a State.successors() para obtener todos los estados válidos alcanzables desde el estado actual.
- **Filtrar y encolar:** para cada sucesor no visitado: marcarlo como visitado y encolarlo junto con el camino extendido (camino_actual + [(sucesor, acción)]).
- **Repetir** desde el paso 2 hasta encontrar el objetivo o agotar la cola.

BFS garantiza la optimalidad porque explora todos los caminos de longitud 1 antes de los de longitud 2, todos los de longitud 2 antes de los de longitud 3, y así sucesivamente. El primer camino que llega al estado objetivo es necesariamente el más corto. En este problema, BFS encuentra la solución en **exactamente 7 pasos**.

Paso 5 — Solución encontrada (7 movimientos óptimos)

Paso	Acción	Estado (F, L, G, C)
0	Inicio	(0, 0, 0, 0) — Izq: Granjero, Lobo, Cabra, Col | Der: vacía
1	Granjero -> con Cabra	(1, 0, 1, 0) — Izq: Lobo, Col | Der: Granjero, Cabra
2	Granjero <- solo	(0, 0, 1, 0) — Izq: Granjero, Lobo, Col | Der: Cabra
3	Granjero -> con Lobo	(1, 1, 1, 0) — Izq: Col | Der: Granjero, Lobo, Cabra
4	Granjero <- con Cabra *	(0, 1, 0, 0) — Izq: Granjero, Cabra, Col | Der: Lobo
5	Granjero -> con Col	(1, 1, 0, 1) — Izq: Cabra | Der: Granjero, Lobo, Col
6	**Granjero <- solo**	**(0, 1, 0, 1) — Izq: Granjero, Cabra | Der: Lobo, Col**
7	**Granjero -> con Cabra**	**(1, 1, 1, 1) — Izq: vacía | Der: Granjero, Lobo, Cabra, Col**

* Paso 4 — movimiento contraintuitivo: tras el paso 3, el estado (1,1,1,0) tiene a lobo y cabra juntos en la orilla derecha. Si el granjero regresara solo, dejaría al lobo con la cabra sin supervisión (estado inválido). La única opción válida es regresar con la cabra, produciendo (0,1,0,0). Hay que 'retroceder' para avanzar.

Diseño OOP — clases implementadas

Función / Módulo	Responsabilidad	Detalle
is_valid(state)	Valida que el estado no viole las restricciones.	Descarta lobo+cabra o cabra+col solos.
successors(state)	Genera los estados alcanzables desde el estado actual.	Hasta 4 acciones: cruzar solo o con lobo/cabra/col.
bfs()	Búsqueda en anchura desde START hasta GOAL.	Cola FIFO; retorna path y actions.
build_graph()	Construye el grafo dirigido completo de estados válidos.	Usa NetworkX DiGraph con aristas etiquetadas.
visualize_steps()	Genera imagen con los 8 paneles de la secuencia.	Guarda granjero_01_pasos.png (150 dpi).
visualize_graph()	Dibuja el grafo de estados con el camino destacado.	Guarda granjero_02_grafo.png (150 dpi).

Visualizaciones — Granjero

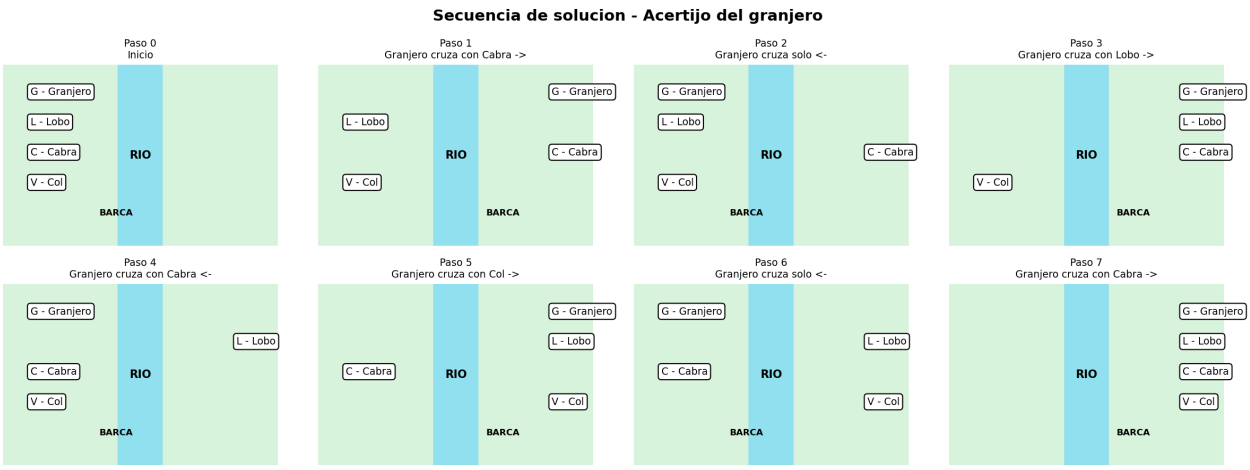
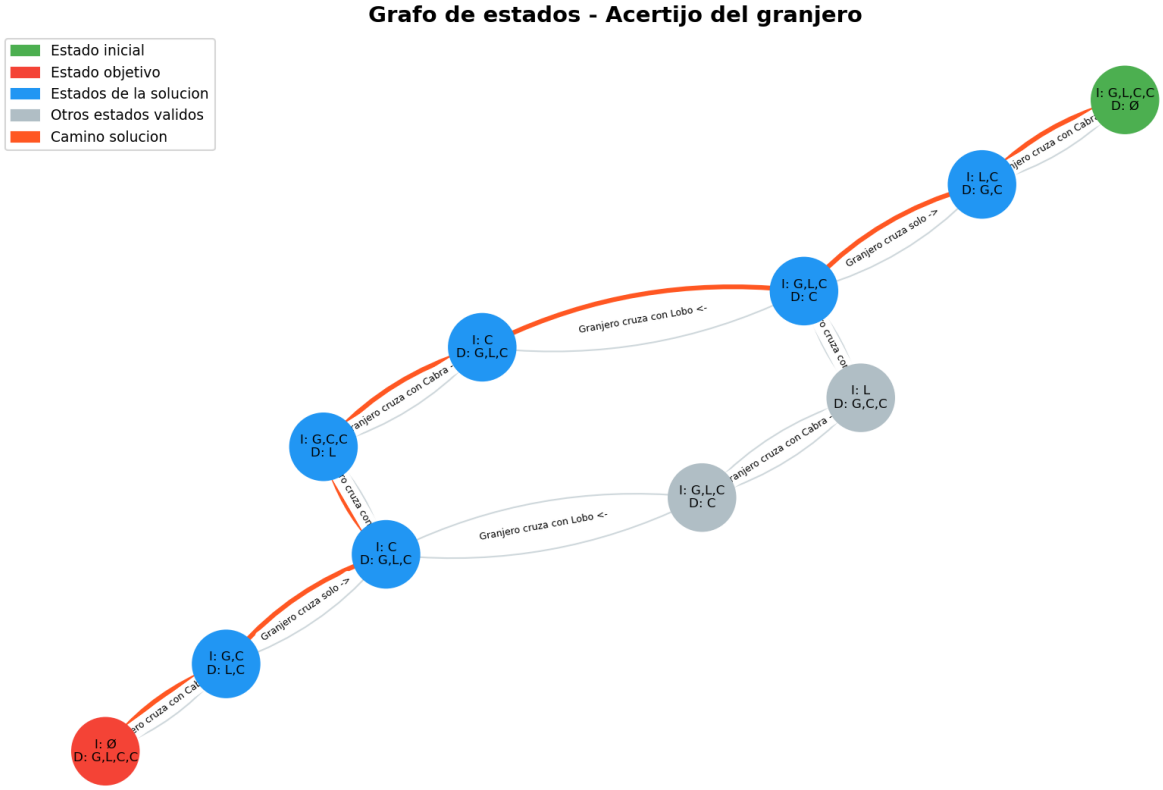


Fig. 6 — Secuencia de solución BFS: 8 paneles (Paso 0 al Paso 7). Cada panel muestra la orilla izquierda (verde), el río (azul) y la orilla derecha (verde). Los elementos se identifican con etiquetas: G-Granjero, L-Lobo, C-Cabra, V-Col. La barca aparece junto al granjero en cada paso. El paso 4 (Granjero regresa con Cabra) es el movimiento contraintuitivo que evita dejar al lobo con la cabra sin supervisión.


Grafo de estados - Acertijo del granjero
Estado inicial
Estado objetivo
Estados de la solucion
Otros estados validos
Camino solucion
I: G,L,C,C
D: Ø
I: L,C
D: G,C
I: G,L,C
D: C
I: C
D: G,L,C
I: L
D: G,C,C
I: G,C,C
D: L
I: G,L,C
D: C
I: C
D: G,L,C
I: G,C
D: L,C
I: Ø
D: G,L,C,C
Granjero cruza solo ->
Granjero cruza con Lobo <-
Granjero cruza con Lobo <-
Granjero cruza solo ->
Granjero cruza con Cabra ->
Granjero cruza con Cabra ->
Granjero cruza con Cabra ->
Granjero cruza con Cabra ->
Granjero cruza solo

enfoque es generalizable a cualquier problema de planificación con restricciones: modelar el estado, definir las transiciones válidas, y aplicar BFS para encontrar el camino más corto.

C. La Torre de Hanoi

Descripción del problema

La Torre de Hanoi es un clásico de la informática teórica. Se tienen **n discos** de distintos tamaños apilados en la Torre A (el más grande abajo), y dos torres vacías B y C. El objetivo es trasladar *todos* los discos a la Torre C siguiendo dos reglas: (1) solo se mueve un disco a la vez, y (2) nunca se coloca un disco más grande sobre uno más pequeño. El problema fue formulado por Édouard Lucas en 1883 y sigue siendo referencia obligatoria en cursos de algoritmos porque exhibe una estructura recursiva perfectamente simétrica.

Estrategia: Divide y Vencerás

La clave es reconocer que mover n discos de A a C es equivalente a tres subproblemas independientes. Esta descomposición es posible porque el disco más grande es completamente independiente de los n–1 discos que están encima de él:

- **Paso 1:** Mover los n–1 discos superiores de A hacia B, usando C como auxiliar.
- **Paso 2:** Mover el disco más grande (disco n) directamente de A hacia C (un solo movimiento).
- **Paso 3:** Mover los n–1 discos de B hacia C, usando A como auxiliar.

Los pasos 1 y 3 son instancias del *mismo problema* con n–1 discos, lo que permite aplicar la misma estrategia recursivamente. El caso base es trivial: con n=1 disco, se mueve directamente de origen a destino sin ninguna complejidad.

Algoritmo recursivo

El pseudocódigo captura la estrategia de manera exacta. La función recibe el número de discos y los nombres de las tres torres (origen, destino, auxiliar):

```
def hanoi(n, origen, destino, auxiliar): # Caso base: mover directamente if n == 1: mover
disco de origen -> destino return # Paso 1: liberar el disco grande hanoi(n-1, origen,
auxiliar, destino) # Paso 2: mover el disco mas grande mover disco de origen -> destino #
Paso 3: reconstruir sobre el disco grande hanoi(n-1, auxiliar, destino, origen)
```

Obsérvese que los roles de las torres **rotan** en cada llamada recursiva: lo que era destino pasa a ser auxiliar y viceversa. Este intercambio es lo que garantiza que nunca se violen las restricciones del problema.

Árbol de recursión (n = 3)

Con n=3, la ejecución genera 7 movimientos. El árbol de llamadas ilustra cómo se descompone el problema:

```
hanoi(3, A, C, B) +--> hanoi(2, A, B, C) | +--> hanoi(1, A, C, B) => mueve disco 1: A -> C
| +--> mover disco 2: A -> B | +--> hanoi(1, C, B, A) => mueve disco 1: C -> B +--> mover
disco 3: A -> C +--> hanoi(2, B, C, A) +--> hanoi(1, B, A, C) => mueve disco 1: B -> A +-->
mover disco 2: B -> C +--> hanoi(1, A, C, B) => mueve disco 1: A -> C
```

Cada nivel del árbol duplica el trabajo: el nivel 0 tiene 1 llamada, el nivel 1 tiene 2, el nivel 2 tiene 4, hasta llegar a las 2■■¹ llamadas base que corresponden a los movimientos reales.

Demostración matemática de la complejidad

Sea T(n) el número de movimientos necesarios para n discos. La recurrencia se formula directamente desde el algoritmo:

```
T(1) = 1 (caso base) T(n) = 2 * T(n-1) + 1 (2 subproblemas de n-1 + 1 movimiento del disco grande)
```

Desarrollando la recurrencia por sustitución repetida:

```
T(n) = 2*T(n-1) + 1 = 2*(2*T(n-2) + 1) + 1 = 4*T(n-2) + 3 = 4*(2*T(n-3) + 1) + 3 = 8*T(n-3) + 7 = ... = 2^(n-1) * T(1) + (2^(n-1) - 1) = 2^(n-1) + 2^(n-1) - 1 = 2^n - 1
```

Este resultado es también un **mínimo absoluto**: se puede demostrar por inducción que cualquier secuencia válida de movimientos requiere al menos 2■–1 pasos, lo que convierte al algoritmo recursivo en óptimo.

Tabla de complejidad: discos vs movimientos

Discos (n)	Movimientos (2■–1)	Tiempo estimado (1 mov/s)
1	1	1 s
2	3	3 s
3	7	7 s
4	15	15 s
5	31	31 s
6	63	1 min 3 s
7	127	2 min 7 s
8	255	4 min 15 s
9	511	8 min 31 s
10	1,023	~17 min
11	2,047	~34 min
12	4,095	~1.1 h
13	8,191	~2.3 h
14	16,383	~4.6 h
15	32,767	~9.1 h
16	65,535	~18.2 h
17	131,071	~36.4 h
18	262,143	~3.0 días
19	524,287	~6.0 días
20	1,048,575	~12.0 días

Diseño orientado a objetos (OOP)

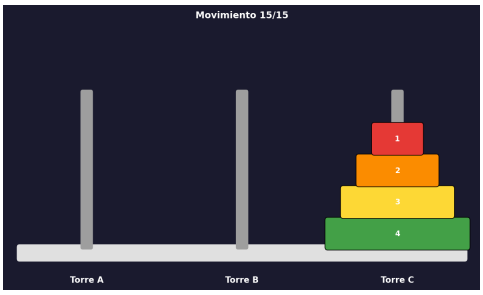
La solución se implementa con cinco clases que siguen el principio de *responsabilidad única*:

Clase	Responsabilidad	Métodos clave
Disk	Entidad: disco con tamaño y color	color (property)
Tower	Pila de discos; valida restricciones	push(disk), pop(), top

HanoiState	Instantánea de las tres torres	snapshot()
HanoiSolver	Ejecuta la recursión y registra movimientos	_recurse(n, towers, src, dst, aux)
HanoiVisualizer	Genera animación y gráficos de análisis	animate(), plot_analysis(), save_all()

La clase **Tower.push()** lanza ValueError si se intenta colocar un disco mayor sobre uno menor, lo que convierte la restricción del puzzle en una invariante verificada en tiempo de ejecución. **HanoiSolver._recurse()** opera directamente sobre las instancias Tower, mientras que **HanoiState** captura una copia profunda de las tres torres en cada paso para permitir la visualización completa.

Visualizaciones — Hanoi



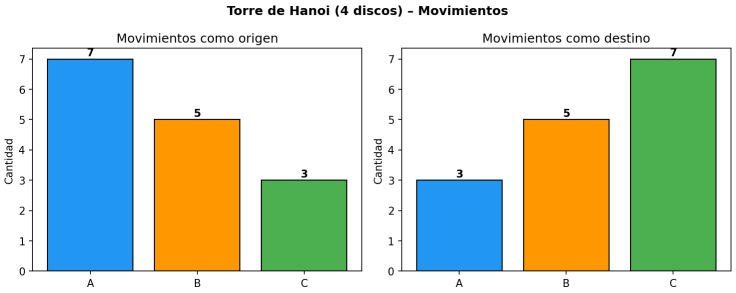


Fig. 8 (izq.) — Estado final con 4 discos: todos apilados en Torre C en orden correcto. Fig. 9 (der.) — Distribución de movimientos: Torre A (origen) genera 8 salidas, Torre C (destino) recibe 8 entradas, Torre B (auxiliar) equilibra el tráfico.

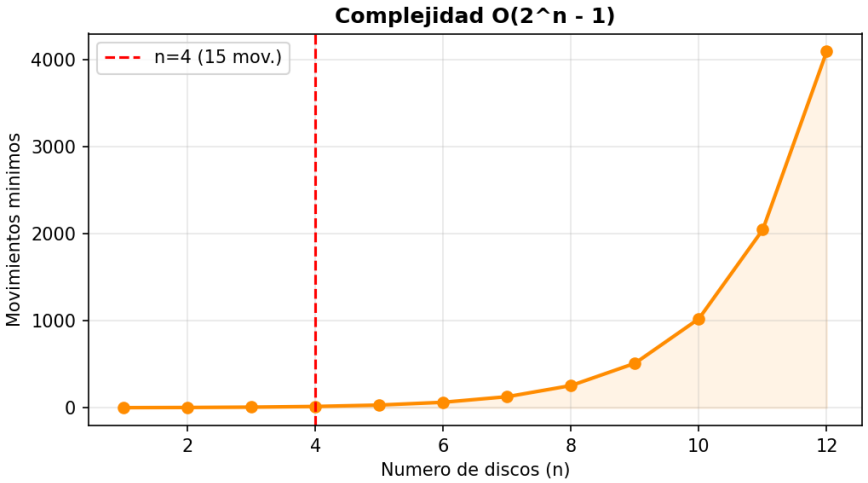


Fig. 10 — Curva de complejidad O(2■ – 1). Con 4 discos se necesitan 15 movimientos; con 10 discos serían 1023; con 20 discos más de 1 millón.

El espacio completo de estados de la Torre de Hanoi

El problema puede también analizarse desde la perspectiva de teoría de grafos. Para **4 discos**, el espacio completo de soluciones se puede representar como un grafo donde cada nodo es una configuración posible de los discos en las tres torres y cada arista es un movimiento legal entre dos configuraciones.

Las dimensiones de este grafo son:

- **81 nodos** — corresponden a todas las configuraciones posibles de los 4 discos en las tres torres. Cada disco puede estar en cualquiera de las 3 torres independientemente, lo que da 3■ = 81 estados totales.

- **120 aristas** — representan los movimientos legales entre configuraciones (solo se puede mover el disco del tope de una torre al tope de otra, siempre que sea más pequeño).
- **Nodo verde** = estado inicial: todos los discos apilados en la Torre A en orden decreciente (disco 4 abajo, disco 1 arriba).
- **Nodo rojo** = estado final: todos los discos apilados en la Torre C en el mismo orden correcto.
- **Camino naranja** = solución óptima de 15 movimientos generada por el algoritmo recursivo de divide y vencerás.

Aunque el problema tiene 81 configuraciones posibles y 120 movimientos legales, el algoritmo recursivo **no explora todo el grafo**. A diferencia del BFS (que sí recorre el grafo completo nivel por nivel), la estrategia de **divide y vencerás** sigue un camino directo y determinístico: en cada nivel de recursión sabe exactamente qué disco mover y hacia dónde, sin necesidad de explorar alternativas. Esto garantiza llegar al objetivo en el **mínimo número de movimientos: 15 para 4 discos**, con una profundidad de pila de recursión de solo n niveles.

Criterio	BFS sobre el grafo	Recursión (Divide y vencerás)
Nodos visitados	Hasta 81 (todos alcanzables)	Solo los 15 del camino óptimo
Garantía de óptimo	Sí (primero en llegar)	Sí (demostrado matemáticamente)
Memoria requerida	O(3^n) — todos los estados	O(n) — pila de recursión
Tiempo de cómputo	O(3^n) — explora el grafo	O(2^n) — solo los movimientos
Aplicable a n grande	No (explosión exponencial)	Sí (eficiente en memoria)

Conclusión — Hanoi

La Torre de Hanoi demuestra tres principios fundamentales de la algoritmia: (1) **Recursión óptima** — el algoritmo encuentra la solución mínima sin necesidad de exploración adicional; (2) **Complejidad exponencial inevitable** — T(n) = 2■–1 no es una debilidad del algoritmo sino un límite inferior del problema; y (3) **Separación de responsabilidades en OOP** — al distribuir la lógica entre Disk, Tower, HanoiSolver y HanoiVisualizer, el código es legible, testeable y extensible (por ejemplo, para 10 discos o para nuevas reglas de movimiento) sin modificar ninguna clase existente.

4. Conclusiones Generales

• **Representación es clave:** La dificultad de cada problema no radica en la implementación sino en elegir la representación correcta. El TSP como grafo ponderado, el Granjero como espacio de estados y Hanoi como árbol de recursión permiten aplicar algoritmos estándar directamente.

• **La IA aprovecha estructura:** ACO no resuelve el TSP por fuerza bruta sino explotando la estructura del problema: las feromonas acumulan experiencia colectiva y la heurística de distancia guía la exploración. Esto es central en IA: usar conocimiento del dominio para reducir el espacio de búsqueda.

• **Visualización como diagnóstico:** Los mapas de feromonas revelan qué aprendió el algoritmo; la curva de convergencia muestra cuándo dejó de mejorar; el grafo de estados del Granjero hace visible por qué algunas rutas son inválidas. Visualizar no es decoración, es comprensión.

• **Complejidad vs. exactitud:** TSP es NP-difícil (no hay solución exacta eficiente), Granjero tiene 16 estados (BFS exacto en ms) y Hanoi tiene solución matemática cerrada. La IA escoge el algoritmo según la naturaleza del problema, no un único enfoque universal.

• **Herramientas Low-Code:** La IA generativa (Claude, ChatGPT, Copilot) permite desarrollar soluciones de calidad académica con ciclos rápidos de prototipado, pero el analista debe validar los resultados, comprender los algoritmos y ajustar los parámetros según el contexto del problema.

Ver también: **ensayo_chips_analogicos.pdf** — La Evolución de la IA con Chips Analógicos (Grupo 5, Abril 2025)